

树状数组

引入

树状数组是一种支持 **单点修改** 和 **区间查询** 的，代码量小的数据结构。

► 什么是「单点修改」和「区间查询」？

普通树状数组维护的信息及运算要满足 **结合律** 且 **可差分**，如加法（和）、乘法（积）、异或等。

- 结合律：，其中 是一个二元运算符。
- 可差分：具有逆运算的运算，即已知 和 可以求出 。

需要注意的是：

- 模意义下的乘法若要可差分，需保证每个数都存在逆元（模数为质数时一定存在）；
- 例如， 这些信息不可差分，所以不能用普通树状数组处理，但是：
 - 使用两个树状数组可以用于处理区间最值，见 [Efficient Range Minimum Queries using Binary Indexed Trees](#)。
 - 本页面也会介绍一种支持不可差分信息查询的， 时间复杂度的拓展树状数组。

事实上，树状数组能解决的问题是线段树能解决的问题的子集：树状数组能做的，线段树一定能做；线段树能做的，树状数组不一定可以。然而，树状数组的代码要远比线段树短，时间效率常数也更小，因此仍有学习价值。

有时，在差分数组和辅助数组的帮助下，树状数组还可解决更强的 **区间加单点值** 和 **区间加区间和** 问题。

树状数组

初步感受

先来举个例子：我们想知道 的前缀和，怎么做？

一种做法是：，需要求 个数的和。

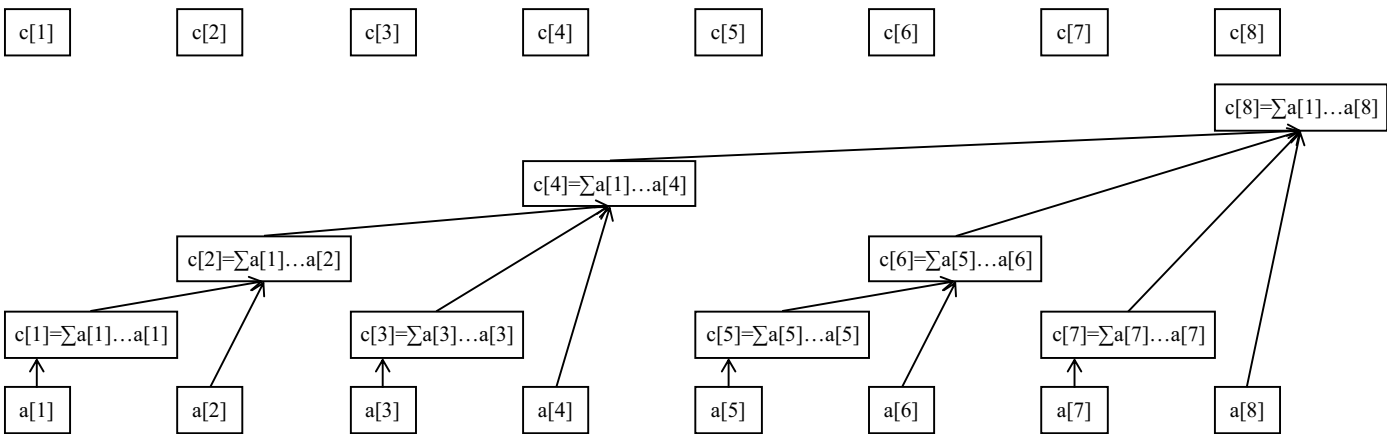
但是如果已知三个数，，， 的和， 的总和， 的总和（其实就是 自己）。你会怎么算？你一定会回答：，只需要求 个数的和。

这就是树状数组能快速求解信息的原因：我们总能将一段前缀 拆成 **不多于 段区间**，使得这 段区间的信息是 **已知的**。

于是，我们只需合并这 段区间的信息，就可以得到答案。相比于原来直接合并 个信息，效率有了很大的提高。

不难发现信息必须满足结合律，否则就不能像上面这样合并了。

下面这张图展示了树状数组的工作原理：



最下面的八个方块代表原始数据数组 。上面参差不齐的方块（与最上面的八个方块是同一个数组）代表数组 的上级—— 数组。

数组就是用来储存原始数组 某段区间的和的，也就是说，这些区间的信息是已知的，我们的目标就是把查询前缀拆成这些小区间。

例如，从图中可以看出：

- 管辖的是；
- 管辖的是；
- 管辖的是；
- 管辖的是；
- 剩下的 管辖的都是 自己（可以看做 的长度为 的小区间）。

不难发现， 管辖的一定是一段右边界是 的区间总信息。我们先不关心左边界，先来感受一下树状数组是如何查询的。

举例：计算 的和。

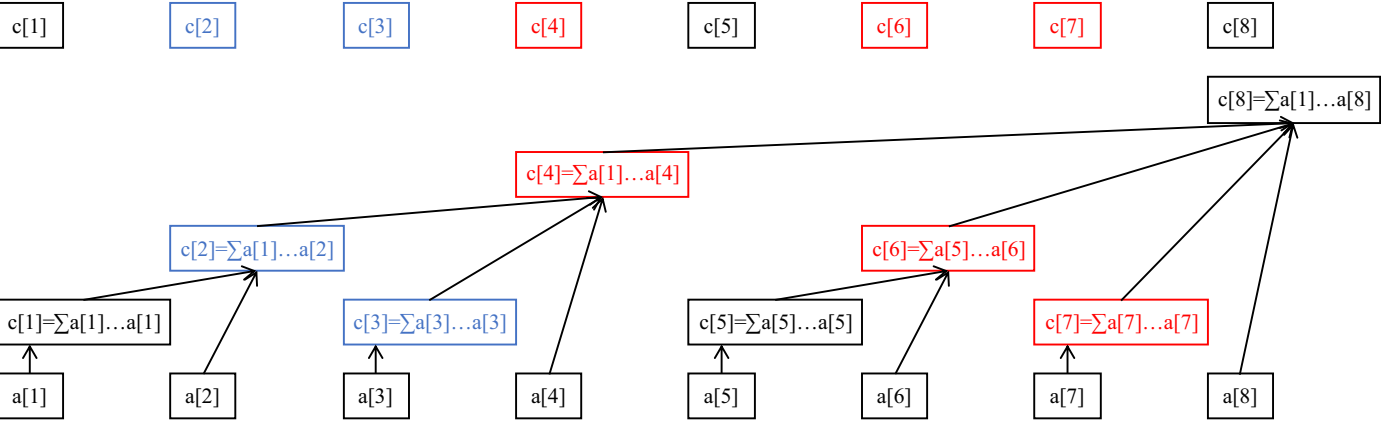
过程：从 开始往前跳，发现 只管辖 这个元素；然后找 ，发现 管辖的是 ，然后跳到 ，发现 管辖的是 这些元素，然后再试图跳到 ，但事实上 不存在，不跳了。

我们刚刚找到的是 ，事实上这就是 拆分出的三个小区间，合并得到答案是 。

举例：计算 的和。

我们还是从 开始跳，跳到 再跳到 。此时我们发现它管理了 的和，但是我们不想要 这一部分，怎么办呢？很简单，减去 的和就行了。

那不妨考虑最开始，就将查询 的和转化为查询 的和，以及查询 的和，最终将两个结果作差。



查询 $\sum a[4] \dots a[7] = (\sum a[1] \dots a[7]) - (\sum a[1] \dots a[3])$

管辖区间

那么问题来了， 管辖的区间到底往左延伸多少？也就是说，区间长度是多少？

树状数组中，规定 管辖的区间长度为 ，其中：

- 设二进制最低位为第 位，则 恰好为 二进制表示中，最低位的 1 所在的二进制位数；
- （ 的管辖区间长度）恰好为 二进制表示中，最低位的 1 以及后面所有 0 组成的数。

举个例子， 管辖的是哪个区间？

因为 ， 其二进制最低位的 1 以及后面的 0 组成的二进制是 1000，即 ，所以 管辖 个 数组中的元素。

因此， 代表 的区间信息。

我们记 二进制最低位 1 以及后面的 0 组成的数为 ，那么 管辖的区间就是 。

这里注意： 指的是不是最低位 1 所在的位数，而是这个 1 和后面所有 0 组成的 。

怎么计算 lowbit？根据位运算知识，可以得到 $\text{lowbit}(x) = x \& -x$ 。

► lowbit 的原理

▼ 实现



C++Python

```
1 int lowbit(int x) {
2     // x 的二进制中，最低位的 1 以及后面所有 0 组成的数。
3     // lowbit(0b01011000) == 0b00001000
4     //      ~~~~~^~~~
5     // lowbit(0b01110010) == 0b00000010
6     //      ~~~~~^~~~
7     return x & -x;
8 }
```

```
1 def lowbit(x):
2     """
3     x 的二进制中，最低位的 1 以及后面所有 0 组成的数。
4     lowbit(0b01011000) == 0b00001000
5     ~~~~~^~~~
6     lowbit(0b01110010) == 0b00000010
7     ~~~~~^~~~
8     """
9     return x & -x
```

区间查询

接下来我们来看树状数组具体的操作实现，先来看区间查询。

回顾查询 的过程，我们是将它转化为两个子过程：查询 和查询 的和，最终作差。

其实任何一个区间查询都可以这么做：查询 的和，就是 的和减去 的和，从而把区间问题转化为前缀问题，更方便处理。

事实上，将有关 的区间询问转化为 和 的前缀询问再差分，在竞赛中是一个非常常用的技巧。

那前缀查询怎么做呢？回顾下查询 的过程：

从 往前跳，发现 只管辖 这个元素；然后找 ，发现 管辖的是 ，然后跳到 ，发现 管辖的是 这些元素，然后再试图跳到 ，但事实上 不存在，不跳了。

我们刚刚找到的 是，事实上这就是 拆分出的三个小区间，合并一下，答案是 。

观察上面的过程，每次往前跳，一定是跳到现区间的左端点的左一位，作为新区间的右端点，这样才能将前缀不重不漏地拆分。比如现在 管的是，下一次就跳到 ，即访问 。

我们可以写出查询 的过程：

- 从 开始往前跳，有 管辖；
- 令，如果 说明已经跳到尽头了，终止循环；否则回到第一步。
- 将跳到的 合并。

实现时，我们不一定要把 都跳出来然后一起合并，可以边跳边合并。

比如我们要维护的信息是和，直接令初始，然后每跳到一个 就，最终 就是所有合并的结果。

▼ 实现



C++Python

```
1 int getsum(int x) { // a[1]..a[x]的和
2     int ans = 0;
3     while (x > 0) {
4         ans = ans + c[x];
5         x = x - lowbit(x);
6     }
7     return ans;
8 }
```

```

1 def getsum(x): # a[1]...a[x]的和
2     ans = 0
3     while x > 0:
4         ans = ans + c[x]
5         x = x - lowbit(x)
6     return ans

```

树状数组与其树形态的性质

在讲解单点修改之前，先讲解树状数组的一些基本性质，以及其树形态来源，这有助于更好理解树状数组的单点修改。

我们约定：

- l ，即，是管辖范围的左端点。
- 对于任意正整数 x ，总能将 x 表示成 2^k 的形式，其中 k 是非负整数。
- 下面「 l 和 r 不交」指 l 的管辖范围和 r 的管辖范围不相交，即 l 和 r 不相交。「 l 包含于 r 」等表述同理。

性质：对于 l 和 r ，要么有 l 和 r 不交，要么有 l 包含于 r 。

► 证明

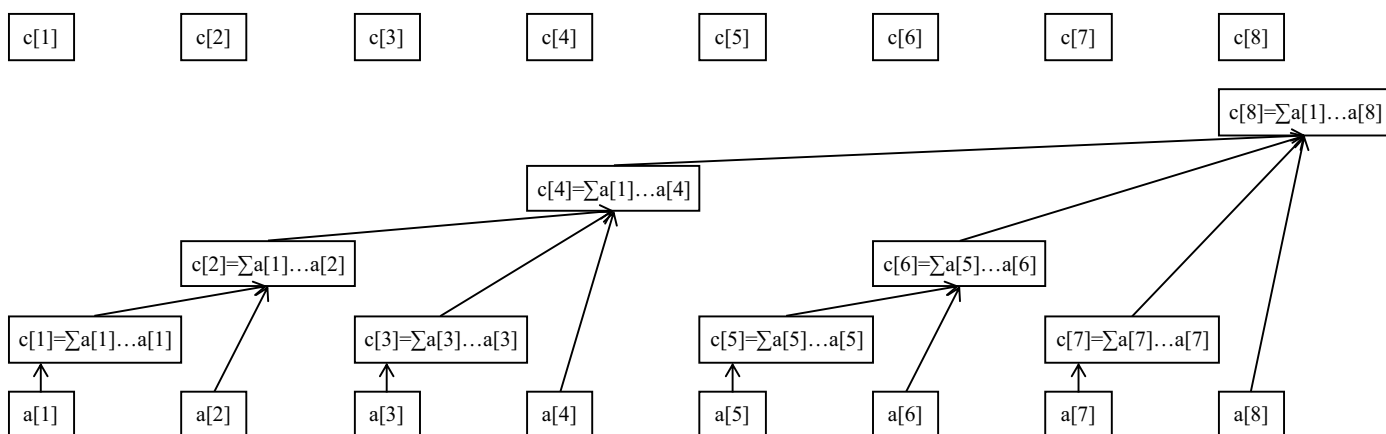
性质：真包含于。

► 证明

性质：对于任意 l ，有 l 和 r 不交。

► 证明

有了这三条性质的铺垫，我们接下来看树状数组的树形态（请忽略 l 向 r 的连边）。



事实上，树状数组的树形态是 l 向 r 连边得到的图，其中 l 是 r 的父亲。

注意，在考虑树状数组的树形态时，我们不考虑树状数组大小的影响，即我们认为这是一棵无限大的树，方便分析。实际实现时，我们只需用到 n 的，其中 n 是原数组长度。

这棵树天然满足了很多美好性质，下面列举若干（设 l 表示 r 的直系父亲）：

- l 是 r 的直系父亲。
- 大于任何一个 l 的后代，小于任何一个 l 的祖先。
- l 点的高度严格小于 r 点的高度。

► 证明

- l 点的高度是 $\log_2 r$ ，即 r 二进制最低位 1 的位数。

► 高度的定义

- 真包含于（性质）。
- 真包含于，其中 l 是 r 的任一祖先（在上一条性质上归纳）。
- 真包含，其中 l 是 r 的任一后代（上面那条性质，颠倒）。

- 对于任意 x ，若 x 不是 u 的祖先，则 u 和 x 不交。

► 证明

- 对于任意 x ，如果 x 不在 u 的子树上，则 u 和 x 不交（上面那条性质， u 颠倒）。
- 对于任意 x ，当且仅当 u 是 x 的祖先， u 真包含于 x （上面几条性质的总结）。这就是树状数组单点修改的核心原理。
- 设 u 的二进制编号为 $\dots 1000$ ，则 u 有三个儿子，二进制编号分别为 $\dots 0111$ 、 $\dots 0110$ 、 $\dots 0100$ 。

► 证明

- u 的所有儿子对应 u 的管辖区间恰好拼接成 u 。
 - 举例：假设 u 的二进制编号为 $\dots 1000$ ，则 u 有三个儿子，二进制编号分别为 $\dots 0111$ 、 $\dots 0110$ 、 $\dots 0100$ 。
 - $c[\dots 0100]$ 表示 $a[\dots 0001 \sim \dots 0100]$ 。
 - $c[\dots 0110]$ 表示 $a[\dots 0101 \sim \dots 0110]$ 。
 - $c[\dots 0111]$ 表示 $a[\dots 0111 \sim \dots 0111]$ 。
 - 不难发现上面是三个管辖区间的并集恰好是 $a[\dots 0001 \sim \dots 0111]$ ，即 u 。

► 证明

单点修改

现在来考虑如何单点修改。

我们的目标是快速正确地维护 c 数组。为保证效率，我们只需遍历并修改管辖了 x 的所有 c ，因为其他的 c 显然没有发生变化。

管辖 x 的 c 一定包含 u （根据性质 1），所以 u 在树状数组树形态上是 x 的祖先。因此我们从 x 开始不断跳父亲，直到跳得超过了原数组长度为止。

设 u 表示 x 的大小，不难写出单点修改 x 的过程：

- 初始令 $x = x$ 。
- 修改 $c[x]$ 。
- 令 $x = x + \text{lowbit}(x)$ ，如果 $x > n$ 说明已经跳到尽头了，终止循环；否则回到第二步。

区间信息和单点修改的种类，共同决定 u 的修改方式。下面给几个例子：

- 若 u 维护区间和，修改种类是将 x 加上 k ，则修改方式则是将所有 c 也加上 k 。
- 若 u 维护区间积，修改种类是将 x 乘上 k ，则修改方式则是将所有 c 也乘上 k 。

然而，单点修改的自由性使得修改的种类和维护的信息不一定是同种运算，比如，若 u 维护区间和，修改种类是将 $c[x]$ 赋值为 k ，可以考虑转化为将 $c[x]$ 加上 $k - c[x]$ 。如果是将 $c[x]$ 乘上 k ，就考虑转化为 $c[x] = c[x] \cdot k$ 。

下面以维护区间和，单点加为例给出实现。

▼ 实现



C++Python

```
1 void add(int x, int k) {
2     while (x <= n) { // 不能越界
3         c[x] = c[x] + k;
4         x = x + lowbit(x);
5     }
6 }
```

```
1 def add(x, k):
2     while x <= n: # 不能越界
3         c[x] = c[x] + k
4         x = x + lowbit(x)
```

建树

也就是根据最开始给出的序列，将树状数组建出来（全部预处理好）。

一般可以直接转化为 次单点修改，时间复杂度（复杂度分析在后面）。

比如给定序列 要求建树，直接看作对 单点加，对 单点加，对 单点加 即可。

也有 的建树方法，见本页面 [建树](#) 一节。

复杂度分析

空间复杂度显然。

时间复杂度：

- 对于区间查询操作：整个 的迭代过程，可看做将 二进制中的所有，从低位到高位逐渐改成 的过程，拆分出的区间数等于 二进制中 的数量（即）。因此，单次查询时间复杂度是；
- 对于单点修改操作：跳父亲时，访问到的高度一直严格增加，且始终有。由于点 的高度是，所以跳到的高度不会超过，所以访问到的 的数量是 级别。因此，单次单点修改复杂度是。

区间加区间和

前置知识：[前缀和 & 差分](#)。

该问题可以使用两个树状数组维护差分数组解决。

考虑序列 的差分数组，其中。由于差分数组的前缀和就是原数组，所以。

一样地，我们考虑将查询区间和通过差分转化为查询前缀和。那么考虑查询 的和，即，进行推导：

观察这个式子，不难发现每个 总共被加了 次。接着推导：

并不能推出 的值，所以要用两个树状数组分别维护 和 的和信息。

那么怎么做区间加呢？考虑给原数组 区间加 给 带来的影响。

因为差分是，

- 多了 而 不变，所以 的值多了。
- 不变而 多了，所以 的值少了。
- 对于不等于 且不等于 的任意，和 要么都没发生变化，要么都加了，还是，所以其它的 均不变。

那就不难想到维护方式了：对于维护 的树状数组，对 单点加， 单点加；对于维护 的树状数组，对 单点加， 单点加。

而更弱的问题，「区间加求单点值」，只需用树状数组维护一个差分数组。询问 的单点值，直接求 的和即可。

这里直接给出「区间加区间和」的代码：

▼ 实现



C++Python

```

1 int t1[MAXN], t2[MAXN], n;
2
3 int lowbit(int x) { return x & (-x); }
4
5 void add(int k, int v) {
6     int v1 = k * v;
7     while (k <= n) {
8         t1[k] += v, t2[k] += v1;
9         // 注意不能写成 t2[k] += k * v, 因为 k 的值已经不是原数组的下标了
10        k += lowbit(k);
11    }
12 }
13
14 int getsum(int *t, int k) {
15     int ret = 0;
16     while (k) {
17         ret += t[k];
18         k -= lowbit(k);
19     }
20     return ret;
21 }
22
23 void add1(int l, int r, int v) {
24     add(l, v), add(r + 1, -v); // 将区间加差分分为两个前缀加
25 }
26
27 long long getsum1(int l, int r) {
28     return (r + 1ll) * getsum(t1, r) - 1ll * l * getsum(t1, l - 1) -
29         (getsum(t2, r) - getsum(t2, l - 1));
30 }

```

```

1 t1 = [0] * MAXN
2 t2 = [0] * MAXN
3 n = 0
4
5
6 def lowbit(x):
7     return x & (-x)
8
9
10 def add(k, v):
11     v1 = k * v
12     while k <= n:
13         t1[k] = t1[k] + v
14         t2[k] = t2[k] + v1
15         k = k + lowbit(k)
16
17
18 def getsum(t, k):
19     ret = 0
20     while k:
21         ret = ret + t[k]
22         k = k - lowbit(k)
23     return ret
24
25
26 def add1(l, r, v):
27     add(l, v)
28     add(r + 1, -v)
29
30
31 def getsum1(l, r):
32     return (
33         (r) * getsum(t1, r)
34         - l * getsum(t1, l - 1)
35         - (getsum(t2, r) - getsum(t2, l - 1))
36     )

```

根据这个原理，应该可以实现「区间乘区间积」，「区间异或一个数，求区间异或值」等，只要满足维护的信息和区间操作是同种

运算即可，感兴趣的读者可以自己尝试。

二维树状数组

单点修改，子矩阵查询

二维树状数组，也被称作树状数组套树状数组，用来维护二维数组上的单点修改和前缀信息问题。

与一维树状数组类似，我们用 c 表示 a 的矩阵总信息，即一个以 $a_{1,1}$ 为右下角，高 n ，宽 m 的矩阵的总信息。

对于单点修改，设：

$lowbit(i)$ 为 i 在树状数组树形态上的第 $lowbit(i)$ 级祖先（第 1 级祖先是自己）。

则只有 $a_{i,j}$ 中的元素管辖 $c_{i,j}$ ，修改 $a_{i,j}$ 时只需修改所有 $c_{i',j'}$ ，其中 $i' = i + lowbit(i)$ ， $j' = j + lowbit(j)$ 。

► 正确性证明

对于查询，我们设：

则合并所有 $c_{i',j'}$ ，其中 $i' = i - lowbit(i)$ ， $j' = j - lowbit(j)$ 。

► 正确性证明

下面给出单点加、查询子矩阵和的代码。

▼ 实现



单点加查询子矩阵和

```
1 void add(int x, int y, int v) {
2     for (int i = x; i <= n; i += lowbit(i)) {
3         for (int j = y; j <= m; j += lowbit(j)) {
4             // 注意这里必须得建循环变量，不能像一维数组一样直接 while (x <= n) 了
5             c[i][j] += v;
6         }
7     }
8 }

1 int sum(int x, int y) {
2     int res = 0;
3     for (int i = x; i > 0; i -= lowbit(i)) {
4         for (int j = y; j > 0; j -= lowbit(j)) {
5             res += c[i][j];
6         }
7     }
8     return res;
9 }
10
11 int ask(int x1, int y1, int x2, int y2) {
12     // 查询子矩阵和
13     return sum(x2, y2) - sum(x2, y1 - 1) - sum(x1 - 1, y2) + sum(x1 - 1, y1 - 1);
14 }
```

子矩阵加，求子矩阵和

前置知识：[前缀和 & 差分](#) 和本页面 [区间加区间和](#) 一节。

和一维树状数组的「区间加区间和」问题类似，考虑维护差分数组。

二维数组上的差分数组是这样的：

。

► 为什么这么定义？

这样以来，对左上角，右下角 的子矩阵区间加，相当于在差分数组上，对 和 分别单点加，对 和 分别单点加。

至于原因，把这四个 分别用定义式表示出来，分析一下每项的变化即可。

举个例子吧，初始差分数组为，给 子矩阵加 后差分数组会变为：

（其中 这个子矩阵恰好是上面位于中心的 大小的矩阵。）

因此，子矩阵加的做法是：转化为差分数组上的四个单点加操作。

现在考虑查询子矩阵和：

对于点，它的二维前缀和可以表示为：

原因就是差分的前缀和的前缀和就是原本的前缀和。

和一维树状数组的「区间加区间和」问题类似，统计 的出现次数，为。

然后接着推导：

所以我们需维护四个树状数组，分别维护，，， 的和信息。

当然了，和一维同理，如果只需要子矩阵加求单点值，维护一个差分数组然后询问前缀和就足够了。

下面给出代码：

▼ 实现

```
1 using ll = long long;
2 ll t1[N][N], t2[N][N], t3[N][N], t4[N][N];
3
4 void add(ll x, ll y, ll z) {
5     for (int X = x; X <= n; X += lowbit(X))
6         for (int Y = y; Y <= m; Y += lowbit(Y)) {
7             t1[X][Y] += z;
8             t2[X][Y] += z * x; // 注意是 z * x 而不是 z * X, 后面同理
9             t3[X][Y] += z * y;
10            t4[X][Y] += z * x * y;
11        }
12 }
13
14 void range_add(ll xa, ll ya, ll xb, ll yb,
15                ll z) { // (xa, ya) 到 (xb, yb) 子矩阵
16     add(xa, ya, z);
17     add(xa, yb + 1, -z);
18     add(xb + 1, ya, -z);
19     add(xb + 1, yb + 1, z);
20 }
21
22 ll ask(ll x, ll y) {
23     ll res = 0;
24     for (int i = x; i; i -= lowbit(i))
25         for (int j = y; j; j -= lowbit(j))
26             res += (x + 1) * (y + 1) * t1[i][j] - (y + 1) * t2[i][j] -
27                 (x + 1) * t3[i][j] + t4[i][j];
28     return res;
29 }
30
31 ll range_ask(ll xa, ll ya, ll xb, ll yb) {
32     return ask(xb, yb) - ask(xb, ya - 1) - ask(xa - 1, yb) + ask(xa - 1, ya - 1);
33 }
```

权值树状数组及应用

我们知道，普通树状数组直接在原序列的基础上构建，表示的就是 的区间信息。

然而事实上，我们还可以在原序列的权值数组上构建树状数组，这就是权值树状数组。

► 什么是权值数组？

运用权值树状数组，我们可以解决一些经典问题。

单点修改，查询全局第 小

在此处只讨论第 小，第 大问题可以通过简单计算转化为第 小问题。

该问题可离散化，如果原序列 值域过大，离散化后再建立权值数组。注意，还要把单点修改中的涉及到的值也一起离散化，不能只离散化原数组 中的元素。

对于单点修改，只需将对原数列的单点修改转化为对权值数组的单点修改即可。具体来说，原数组 从 修改为 ，转化为对权值数组的单点修改就是 单点减， 单点加。

对于查询第 小，考虑二分，查询权值数组中的前缀和，找到 使得 的前缀和 而 的前缀和，则第 大的数是（注：这里认为 的前缀和是）。

这样做时间复杂度是 的。

考虑用倍增替代二分。

设，，枚举 从 降为：

- 查询权值数组中的区间和。
- 如果，扩展成功，，；否则扩展失败，不操作。

这样得到的 是满足 前缀和 的最大值，所以最终 就是答案。

看起来这种方法时间效率没有任何改善，但事实上，查询 的区间和只需访问 的值即可。

原因很简单，考虑，它一定是，因为 之前只累加过 满足。因此 表示的区间就是。

如此一来，时间复杂度降低为。

▼ 实现



C++Python

```
1 // 权值树状数组查询第 k 小
2 int kth(int k) {
3     int sum = 0, x = 0;
4     for (int i = log2(n); ~i; --i) {
5         x += 1 << i; // 尝试扩展
6         if (x >= n || sum + t[x] >= k) // 如果扩展失败
7             x -= 1 << i;
8         else
9             sum += t[x];
10    }
11    return x + 1;
12 }
```

```

1 # 权值树状数组查询第 k 小
2 def kth(k):
3     sum = 0
4     x = 0
5     i = int(log2(n))
6     while ~i:
7         x = x + (1 << i) # 尝试扩展
8         if x >= n or sum + t[x] >= k: # 如果扩展失败
9             x = x - (1 << i)
10        else:
11            sum = sum + t[x]
12        i = i - 1
13    return x + 1

```

全局逆序对（全局二维偏序）

相关阅读和参考实现：[逆序对](#)

全局逆序对也可以用权值树状数组巧妙解决。问题是这样的：给定长度为 n 的序列，求其中满足 $i < j$ 且 $a_i > a_j$ 的数对 (i, j) 的数量。

该问题可离散化，如果原序列值域过大，离散化后再建立权值数组。

我们考虑从 n 到 1 倒序枚举，作为逆序对中第一个元素的索引，然后计算有多少个 j 满足 $a_j > a_i$ ，最后累计答案即可。

事实上，我们只需要这样做（设当前为 i ）：

- 查询 a_i 的前缀和，即为左端点为 i 的逆序对数量。
- 自增；

原因十分自然：出现在 i 中的元素一定比当前的 a_i 小，而 i 的倒序枚举，自然使得这些已在权值数组中的元素，在原数组上的索引 j 大于当前遍历到的索引 i 。

用例子说明，。

按照 扫：

- $i=5$ ，查询 a_5 前缀和，为 0 ，自增，。
- $i=4$ ，查询 a_4 前缀和，为 1 ，自增，。
- $i=3$ ，查询 a_3 前缀和，为 2 ，自增，。
- $i=2$ ，查询 a_2 前缀和，为 4 ，自增，。
- $i=1$ ，查询 a_1 前缀和，为 6 ，自增，。

所以最终答案为 6 。

注意到，遍历后的查询和自增的两个步骤可以颠倒，变成先自增再查询，不影响答案。两个角度来解释：

- 对 a_i 的修改不影响对 a_j 的查询。
- 颠倒后，实质是在查询 a_j 且 $j > i$ 的数对数量，而 i 时不存在，所以 i 相当于 0 ，所以这与原来的逆序对问题是等价的。

如果查询非严格逆序对（ $a_i \geq a_j$ ）的数量，那就要改为查询 a_j 的和，这时就不能颠倒两步了，还是两个角度来解释：

- 对 a_i 的修改影响对 a_j 的查询。
- 颠倒后，实质是在查询 a_j 且 $j > i$ 的数对数量，而 i 时恒有 $a_i \geq a_i$ ，所以 i 不相当于 0 ，与原问题不等价。

如果查询 $a_i > a_j$ 的数对数量，那这两步就需要颠倒了。

另外，对于原逆序对问题，还有一种做法是正序枚举，查询有多少 j 满足 $a_j > a_i$ 。做法如下（设 i ）：

- 查询 a_i 是第几小（即 a_i 的值域（或离散化后的值域））的区间和。
- 将 a_i 自增。

原因：出现在 i 中的元素一定比当前的 a_i 大，而 i 的正序枚举，自然使得这些已在权值数组中的元素，在原数组上的索引 j 小于当前遍历到的索引 i 。

此外，逆序对的计数还可以通过 [归并排序](#) 解决。这一方法可以避免离散化。时间复杂度同样为 $O(n \log n)$ 。两种算法的参考实现都在 [逆序对](#) 章节。

树状数组维护不可差分信息

比如维护区间最值等。

注意，这种方法虽然码量小，但单点修改和区间查询的时间复杂度均为 $O(n \log n)$ ，比使用线段树的时间复杂度 劣。

区间查询

我们还是基于之前的思路，从 r 沿着 r 一直向前跳，但是我们不能跳到 r 的左边。

因此，如果我们跳到了 r ，先判断下一次要跳到的 $r - \text{lowbit}(r)$ 是否小于 l ：

- 如果小于 l ，我们直接把 r 合并到总信息里，然后跳到 $r - \text{lowbit}(r)$ 。
- 如果大于等于 l ，说明没越界，正常合并 r ，然后跳到 $r - \text{lowbit}(r)$ 即可。

下面以查询区间最大值为例，给出代码：

▼ 实现

```
1 int getmax(int l, int r) {
2     int ans = 0;
3     while (r >= l) {
4         ans = max(ans, a[r]);
5         --r;
6         for (; r - lowbit(r) >= l; r -= lowbit(r)) {
7             // 注意，循环条件不要写成 r - lowbit(r) + 1 >= l
8             // 否则 l = 1 时，r 跳到 0 会死循环
9             ans = max(ans, C[r]);
10        }
11    }
12    return ans;
13 }
```

可以证明，上述算法的时间复杂度是 $O(n \log n)$ 。

► 时间复杂度证明

单点更新

▼ 注

请先理解树状数组树形态的以下两条性质，再学习本节。

- 设 r ，则其儿子数量为 $\text{lowbit}(r)$ ，编号分别为 $r - \text{lowbit}(r) + 1, \dots, r$ 。
- r 的所有儿子对应 $r - \text{lowbit}(r)$ 的管辖区间恰好拼接成 $[r - \text{lowbit}(r), r - 1]$ 。

关于这两条性质的含义及证明，都可以在本页面的 [树状数组与其树形态的性质](#) 一节找到。

更新 r 后，我们只需要更新满足在树状数组树形态上，满足 r 是 r 的祖先的 r 。

对于最值（以最大值为例），一种常见的错误想法是，如果 r 修改成 r' ，则将所有 r 更新为 r' 。下面是一个反例：中将 r 修改成 r' ，最大值是 r' ，但按照上面的修改这样会得到 r' 。将 r 直接修改为 r' 也是错误的，一个反例是，将上面例子中的 r 修改为 r' 。

事实上，对于不可差分信息，不存在通过 r 直接修改 r' 的方式。这是因为修改本身就相当于是把旧数从原区间「移除」，然后加入一个新数。「移除」时对区间信息的影响，相当于做「逆运算」，而不可差分信息不存在「逆运算」，所以无法直接修改。

换句话说，对每个受影响的 r ，这个区间的信息我们必定要重构了。

考虑 r 的儿子们，它们的信息一定是正确的（因为我们先更新儿子再更新父亲），而这些儿子又恰好组成了这一段管辖区间，那再合并一个单点 r 就可以合并出 r ，也就是 r' 了。这样，我们能至多 $\log n$ 个区间重构合并出每个需要修改的 r 。

▼ 实现

```

1 void update(int x, int v) {
2     a[x] = v;
3     for (int i = x; i <= n; i += lowbit(i)) {
4         // 枚举受影响的区间
5         C[i] = a[i];
6         for (int j = 1; j < lowbit(i); j *= 2) {
7             C[i] = max(C[i], C[i - j]);
8         }
9     }
10 }

```

容易看出上述算法时间复杂度为 $O(n^2)$ 。

建树

可以考虑拆成 n 个单点修改，建树。

也有 的建树方法，见本页面 [建树](#) 一节的方法一。

Tricks

建树

以维护区间和为例。

方法一：

每一个节点的值是由所有与自己直接相连的儿子的值求和得到的。因此可以倒着考虑贡献，即每次确定完儿子的值后，用自己的值更新自己的直接父亲。

▼ 实现



C++Python

```

1 //  $\Theta(n)$  建树
2 void init() {
3     for (int i = 1; i <= n; ++i) {
4         t[i] += a[i];
5         int j = i + lowbit(i);
6         if (j <= n) t[j] += t[i];
7     }
8 }

```

```

1 #  $\Theta(n)$  建树
2 def init():
3     for i in range(1, n + 1):
4         t[i] = t[i] + a[i]
5         j = i + lowbit(i)
6         if j <= n:
7             t[j] = t[j] + t[i]

```

方法二：

前面讲到 表示的区间是 $[i, i + \text{lowbit}(i) - 1]$ ，那么我们可以先预处理一个 前缀和数组，再计算 数组。

▼ 实现



C++Python

```

1 //  $\Theta(n)$  建树
2 void init() {
3     for (int i = 1; i <= n; ++i) {
4         t[i] = sum[i] - sum[i - lowbit(i)];
5     }
6 }

1 #  $\Theta(n)$  建树
2 def init():
3     for i in range(1, n + 1):
4         t[i] = sum[i] - sum[i - lowbit(i)]

```

时间戳优化

对付多组数据很常见的技巧。若每次输入新数据都暴力清空树状数组，就可能会造成超时。因此使用 标记，存储当前节点上次使用时间（即最近一次是被第几组数据使用）。每次操作时判断这个位置 中的时间和当前时间是否相同，就可以判断这个位置应该是还是数组内的值。

▼ 实现



C++Python

```

1 // 时间戳优化
2 int tag[MAXN], t[MAXN], Tag;
3
4 void reset() { ++Tag; }
5
6 void add(int k, int v) {
7     while (k <= n) {
8         if (tag[k] != Tag) t[k] = 0;
9         t[k] += v, tag[k] = Tag;
10        k += lowbit(k);
11    }
12 }
13
14 int getsum(int k) {
15     int ret = 0;
16     while (k) {
17         if (tag[k] == Tag) ret += t[k];
18         k -= lowbit(k);
19     }
20     return ret;
21 }

```

```
1 # 时间戳优化
2 tag = [0] * MAXN
3 t = [0] * MAXN
4 Tag = 0
5
6
7 def reset():
8     Tag = Tag + 1
9
10
11 def add(k, v):
12     while k <= n:
13         if tag[k] != Tag:
14             t[k] = 0
15             t[k] = t[k] + v
16             tag[k] = Tag
17             k = k + lowbit(k)
18
19
20 def getsum(k):
21     ret = 0
22     while k:
23         if tag[k] == Tag:
24             ret = ret + t[k]
25         k = k - lowbit(k)
26     return ret
```

例题

- [树状数组 1：单点修改，区间查询](#)
- [树状数组 2：区间修改，单点查询](#)
- [树状数组 3：区间修改，区间查询](#)
- [二维树状数组 1：单点修改，区间查询](#)
- [二维树状数组 2：区间修改，单点查询](#)
- [二维树状数组 3：区间修改，区间查询](#)

本页面最近更新：2025/3/30 18:59:31, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [ananbaobeichicun](#), [HeRaNO](#), [HowieHz](#), [iamtwz](#), [lr1d](#), [ouuan](#), [ranwen](#), [shuzhouliu](#), [wangdehu](#), [Xeonacid](#), [Zhoier](#), [aofall](#), [AtomAlpaca](#), [c-forrest](#), [chenryang](#), [Chrogeek](#), [ChungZH](#), [CoelacanthusHex](#), [corchis-S](#), [countercurrent-time](#), [david-why](#), [dbxxx-ac](#), [dbxxx-oi](#), [Early0v0](#), [Enter-tainer](#), [FinParker](#), [Great-designer](#), [H-J-Granger](#), [ksyx](#), [LiuZengqiang](#), [Marcythm](#), [mcendu](#), [megakite](#), [Menci](#), [NachtgeistW](#), [Nemodontcry](#), [Persdre](#), [RuiYu2021](#), [shawlleyw](#), [sshwy](#), [SukkaW](#), [Suyun514](#), [Tiphereth-A](#), [Weijun-Lin](#), [Ycrpro](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用